

Sequence Models, Attention & Transformers

Overview

Many AI tasks are sequential: language, code, logs, time series, user sessions, biological sequences. This chapter explains how sequence modeling evolved from recurrent networks to attention and transformers.

Goal:

- understand the conceptual path to modern LLMs,
- recognize engineering trade-offs across sequence model families,
- prepare for Lesson 5 (generative models and LLM systems).

Sequence Modeling Challenges

Key difficulties in sequence tasks:

- variable-length inputs,
- long-range dependencies,
- order sensitivity,
- computational efficiency at scale.

A good sequence model must balance context capacity, parallelism, and stability.

RNN and LSTM Intuition

RNN Core Idea

RNNs process one token/time-step at a time while carrying hidden state forward.

Pros:

- simple recurrence mechanism,
- natural for streaming data.

Limitations:

- vanishing/exploding gradients over long sequences,
- limited parallelism during training.

LSTM/GRU Improvements

Gates control information flow and retention, improving long-range learning compared to vanilla RNNs.

Still, recurrence remains sequential and hard to scale for very long contexts.

Attention Mechanism

Attention allows each token representation to selectively focus on other tokens.

High-level formula:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

Interpretation:

- Q (query): what current token seeks,
- K (key): what each token offers,
- V (value): content to aggregate.

Benefits:

- direct long-range interaction,
- better gradient paths,
- improved interpretability relative to pure recurrence.

Transformer Fundamentals

Transformers replace recurrence with stacks of self-attention + feed-forward blocks.

Core components:

- token embeddings,
- positional encoding,
- multi-head self-attention,
- residual connections + layer normalization,
- feed-forward sublayers.

Why transformers scaled:

- high parallelism in training,
- strong long-context modeling,
- adaptable pretraining/fine-tuning paradigm.

Practical Model Family Trade-offs

- RNN/LSTM: good for constrained streaming and smaller setups.
- **Transformer encoder**: classification/retrieval/representation tasks.
- **Transformer decoder**: autoregressive generation (LLMs).
- **Encoder-decoder**: sequence-to-sequence tasks (translation/summarization).

Selection depends on latency, context length, memory budget, and task objective.

Engineering Bridge to GenAI and LLM Systems

This chapter connects directly to later modules:

- Lesson 5: LLM foundations and prompt engineering.
- Lesson 6/12: LLMOps lifecycle and observability.
- Lesson 7/13: agentic systems and safety guardrails.

Practical bridge concepts:

- tokenization and context windows,
- inference cost vs quality,
- retrieval augmentation for grounded outputs,
- evaluation beyond perplexity (task quality + safety + business metrics).

Project Prompts & Practice Exercises

1. Train a toy character-level RNN and compare with a small transformer on next-token prediction.
2. Implement simple attention visualization on a short sentence pair.
3. Build a mini seq2seq summarization demo using a small pretrained transformer.
4. Create a cost-quality checklist for choosing model/context length in an LLM application.

Interview Questions & Answers

1. **Q:** Why are sequence models needed in NLP?
A: Meaning depends on token order and context dependencies.

2. **Q:** Main limitation of vanilla RNNs?
A: Difficulty learning long-range dependencies due to gradient issues.
3. **Q:** What do LSTM gates do?
A: Control what information to keep, update, or forget over time.
4. **Q:** Why was attention a breakthrough?
A: It enables direct token-to-token dependency modeling without long recurrent chains.
5. **Q:** What is self-attention?
A: Attention where query/key/value come from the same sequence.
6. **Q:** Why is positional encoding required in transformers?
A: Self-attention alone is order-agnostic; position signals restore sequence order information.
7. **Q:** Multi-head attention intuition?
A: Different heads learn complementary relational patterns in parallel.
8. **Q:** Encoder vs decoder transformer?
A: Encoders build contextual representations; decoders generate autoregressively.
9. **Q:** Why did transformers dominate LLMs?
A: Scalable training parallelism and strong long-context performance.
10. **Q:** What is causal masking?
A: Restricting each token to attend only to past tokens during autoregressive training.
11. **Q:** Why are context windows operationally important?
A: They affect capability, latency, memory use, and inference cost.
12. **Q:** When might RNNs still be useful?
A: Lightweight streaming or constrained environments.
13. **Q:** What metric limitations exist for language modeling?
A: Perplexity does not fully capture factuality, usefulness, or safety.
14. **Q:** How does this chapter help with RAG systems later?
A: Understanding attention/context helps design retrieval and prompt context strategies.
15. **Q:** What is the practical first step before deploying transformer apps?
A: Define evaluation set and cost/latency constraints alongside quality metrics.

Bridge to Lesson 5

You now have the deep learning foundation needed for generative systems. Lesson 5 extends these ideas into VAEs/GANs, autoregressive generation, diffusion models, LLM prompting, and RAG/agent architectures.